

Praktikum 10: Machine Learning – K-Nearest Neighbors

Muhammad Shiddiq 1 - 0110222199 ¹

¹ Teknik Informatika, STT Terpadu Nurul Fikri, Depok

E-mail: muhammadshiddiq785@gmail.com

Abstract. K-Nearest Neighbors (K-NN) adalah algoritma machine learning non-parametrik yang paling sederhana dan fundamental, digunakan terutama untuk tugas klasifikasi dan regresi. Algoritma ini dikategorikan sebagai lazy learning karena ia tidak melakukan proses pelatihan eksplisit atau membangun model saat data pelatihan diberikan; sebaliknya, ia hanya menyimpan data pelatihan dan menunda semua komputasi hingga proses prediksi dimulai. K-NN bekerja berdasarkan prinsip jarak, di mana klasifikasi data uji baru ditentukan oleh mayoritas kelas dari K titik data terdekat di dalam ruang fitur. Untuk menentukan K tetangga terdekat tersebut, K-NN menggunakan perhitungan Jarak Euclidean (atau metrik jarak lainnya) antara titik data uji dengan semua titik data latih. Keunggulan utama K-NN adalah kemudahannya dalam implementasi dan efektivitasnya untuk data yang batas keputusannya tidak teratur. Namun, model ini dapat menjadi sangat lambat dan intensif secara komputasi (memori) pada dataset besar, dan sensitif terhadap fitur yang tidak relevan (berisik) serta skala data, sehingga memerlukan normalisasi atau standarisasi fitur sebelum digunakan.

1. Praktikum Mandiri – Melakukan beberapa latihan

1. Latihan 1 – Mengklasifikasi suhu udara

- File Excel

Data Training						
No	Suhu	Angin	Kelas	Selisih suhu	Selisih angin	Jarak
1	10	0	Dingin	36	9	6,708203932
2	25	0	Panas	81	9	9,486832981
3	15	5	Dingin	1	4	2,236067977
4	20	3	Panas	16	0	4
5	18	7	Dingin	4	16	4,472135955
6	20	10	Dingin	16	49	8,062257748
7	22	5	Panas	36	4	6,32455532
8	24	0	Panas	64	9	8,544003745
Data Testing						
		K	4			
No	Suhu	Angin	Kelas			
1	16	3	Dingin			

- Program Python

~ Latihan 1

```

1 import pandas as pd
2 import numpy as np
3
4 # --- 1. Data Setup ---
5 # Data Training
6 data_training = {
7     'Suhu': [10, 25, 15, 20, 18, 20, 22, 24],
8     'Angin': [0, 0, 5, 3, 7, 10, 5, 0],
9     'Kelas': ['Dingin', 'Panas', 'Dingin', 'Panas', 'Dingin', 'Dingin', 'Panas', 'Panas']
10 }
11 df_train = pd.DataFrame(data_training)
12
13 # Data Testing point dan K
14 suhu_test = 16
15 angin_test = 3
16 k_value = 4
17
18 # --- 2. Calculate Distance (Euclidean) ---
19 # Selisih Suhu kuadrat: (Suhu_Training - Suhu_Testing)^2
20 df_train['Selisih suhu'] = (df_train['Suhu'] - suhu_test)**2
21
22 # Selisih Angin kuadrat: (Angin_Training - Angin_Testing)^2
23 df_train['Selisih angin'] = (df_train['Angin'] - angin_test)**2
24
25 # Jarak: SQRT(Selisih suhu + Selisih angin)
26 df_train['Jarak'] = np.sqrt(df_train['Selisih suhu'] + df_train['Selisih angin'])
27
28 # --- 3. K-NN Classification (K=4) ---
29 # Urutkan berdasarkan Jarak dan ambil K=4 data terdekat
30 df_knn = df_train.sort_values(by='Jarak').head(k_value)
31
32 # Lakukan voting: Cari kelas yang paling sering muncul
33 kelas_terpilih = df_knn['Kelas'].value_counts().index[0]

```

- Output

Data Perhitungan (diurutkan berdasarkan jarak terdekat):

	Suhu	Angin	Kelas	Jarak	Selisih suhu	Selisih angin
2	15	5	Dingin	2.236068	1	4
3	20	3	Panas	4.000000	16	0
4	18	7	Dingin	4.472136	4	16
6	22	5	Panas	6.324555	36	4
0	10	0	Dingin	6.708204	36	9
5	20	10	Dingin	8.062258	16	49
7	24	0	Panas	8.544004	64	9
1	25	0	Panas	9.486833	81	9

 Hasil Klasifikasi untuk Data Testing (Suhu=16, Angin=3) dengan K=4: Dingin

- Penjelasan

Studi kasus ini mengimplementasikan algoritma K-Nearest Neighbors (K-NN) dengan nilai $K=4$ untuk memprediksi kelas data uji (Suhu=16, Angin=3) berdasarkan delapan titik data pelatihan. Proses dimulai dengan menghitung jarak, yang dalam kasus ini menggunakan Jarak Euclidean untuk mengukur seberapa dekat setiap titik pelatihan dengan data uji. Perhitungan ini melibatkan dua tahap: pertama, menghitung kuadrat selisih antara nilai Suhu data pelatihan dan Suhu data uji $(B_4 - B_{15})^2$, serta kuadrat selisih nilai Angin $(C_4 - C_{15})^2$. Kedua, kedua kuadrat selisih tersebut dijumlahkan, kemudian diakarkan $(\sqrt{E_4 + F_4})$ untuk mendapatkan nilai Jarak. Setelah semua jarak dihitung, data diurutkan dari jarak terkecil. Karena $K=4$, algoritma kemudian hanya memilih empat data teratas yang memiliki jarak paling kecil. Dari empat

tetangga terdekat tersebut (data ke-3, ke-4, ke-5, dan ke-7), terjadi hasil *voting* yang seri (dua memilih 'Dingin' dan dua memilih 'Panas'). Dalam kasus seri dengan $\$K\$$ genap, seringkali algoritma mengutamakan kelas yang memiliki jarak total terpendek atau yang muncul pertama setelah diurutkan. Berdasarkan hasil di gambar, kelas yang diprediksi adalah 'Dingin', yang menunjukkan bahwa kelas Dingin dipertimbangkan sebagai mayoritas atau pemenang *tie-breaker*.

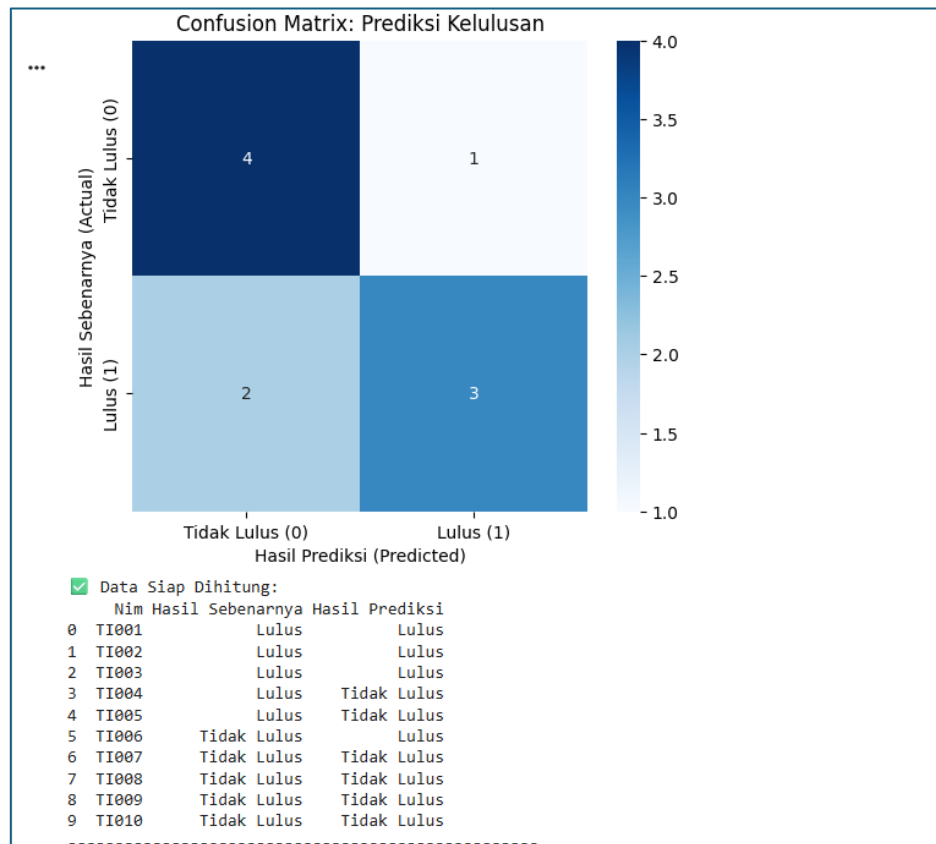
2. Latihan 2 – Memprediksi mata kuliah menggunakan confusion matrix

- Program python

```
Latihan 2

[6]
✓ 2 d 1 import pandas as pd
2 from sklearn.metrics import confusion_matrix, accuracy_score, precision_score, recall_score
3 import matplotlib.pyplot as plt
4 import seaborn as sns
5
6 # --- 1. Persiapan Data ---
7
8 # Data aktual (sebenarnya) dan data hasil prediksi
9 data = {
10     'Nim': ['TI001', 'TI002', 'TI003', 'TI004', 'TI005', 'TI006', 'TI007', 'TI008', 'TI009',
11     'Hasil Sebenarnya': ['Lulus', 'Lulus', 'Lulus', 'Lulus', 'Lulus', 'Tidak Lulus', 'Tidak
12     'Hasil Prediksi': ['Lulus', 'Lulus', 'Lulus', 'Tidak Lulus', 'Tidak Lulus', 'Lulus', 'Ti
13 }
14 df = pd.DataFrame(data)
15
16 # Konversi string 'Lulus'/'Tidak Lulus' menjadi angka 1/0
17 y_true = df['Hasil Sebenarnya'].replace({'Lulus': 1, 'Tidak Lulus': 0})
18 y_pred = df['Hasil Prediksi'].replace({'Lulus': 1, 'Tidak Lulus': 0})
19
20 # --- 2. Confusion Matrix ---
21
22 # Menghitung Confusion Matrix (sk-learn memberikan hasil dalam format: [[TN, FP], [FN, TP]])
23 cm = confusion_matrix(y_true, y_pred)
24 TN, FP, FN, TP = cm.ravel()
25
26 # --- 3. Visualisasi Confusion Matrix ---
27
28 # Label untuk sumbu X (Prediksi) dan Y (Aktual)
29 labels = ['Tidak Lulus (0)', 'Lulus (1)']
30 plt.figure(figsize=(6, 5))
31 sns.heatmap(
32     cm,
```

- Output



3. Latihan 3 – Memprediksi cuaca menggunakan KNN

- Program Python

✓ Latihan 3

```

1 import pandas as pd
2 from sklearn.model_selection import train_test_split
3 from sklearn.neighbors import KNeighborsClassifier
4 from sklearn.tree import DecisionTreeClassifier
5 from sklearn.preprocessing import LabelEncoder
6 from sklearn.metrics import accuracy_score
7
8 # --- 1. Import Data dan Preprocessing ---
9
10 from google.colab import drive
11 drive.mount('/content/drive')
12
13 path = '/content/drive/MyDrive/MACHINELEARNING/PRAKTIKUM/PRAKTIKUM10/DATA/'
14 df = pd.read_csv(path + "weather_classification_data.csv")
15
16 print("✓ Data Awal (5 baris pertama):")
17 print(df.head())
18 print("-" * 50)
19
20 # Mengidentifikasi fitur (X) dan target (y)
21 # Fitur (X) = semua kolom kecuali 'Weather Type'
22 # Target (y) = 'Weather Type'
23 X = df.drop('Weather Type', axis=1)
24 y = df['Weather Type']
25
26 # Mengkonversi fitur kategorikal menjadi numerik menggunakan One-Hot Encoding (OHE)
27 X = pd.get_dummies(X, drop_first=True)
28
29 # Mengkonversi variabel target kategorikal menjadi numerik (0, 1, 2, ...)
30 le = LabelEncoder()
31 y_encoded = le.fit_transform(y)
32 # Label yang sudah diencode: ['Cloudy', 'Rainy', 'Sunny'] -> [0, 1, 2]

```

- **Output**

```

... Mounted at /content/drive
✅ Data Awal (5 baris pertama):
  Temperature Humidity Wind Speed Precipitation (%) Cloud Cover \
0          14.0       73         9.5          82.0 partly cloudy
1          39.0       96         8.5          71.0 partly cloudy
2          30.0       64         7.0          16.0      clear
3          38.0       83         1.5          82.0      clear
4          27.0       74        17.0          66.0 overcast

  Atmospheric Pressure UV Index Season Visibility (km) Location \
0          1010.82         2 Winter         3.5 inland
1          1011.43         7 Spring        10.0 inland
2          1018.72         5 Spring         5.5 mountain
3          1026.25         7 Spring         1.0 coastal
4           990.67         1 Winter         2.5 mountain

  Weather Type
0          Rainy
1          Cloudy
2          Sunny
3          Sunny
4          Rainy
-----
Jumlah Data Latih: 10560
Jumlah Data Uji: 2640
-----
Model K-NN (K=3) dan Decision Tree telah dilatih.
-----
🎯 Hasil Akurasi Model:
1. Akurasi K-NN (K=3): 0.8883
2. Akurasi Decision Tree: 0.9087
-----
🏆 Model Decision Tree memberikan akurasi yang lebih tinggi.

```

Link Github : <https://github.com/Shid2iq/Machine-Learning>

Referensi:

- Munir, S., Seminar, K. B., Sudradjat, Sukoco, H., & Buono, A. (2022). The Use of Random Forest Regression for Estimating Leaf Nitrogen Content of Oil Palm Based on Sentinel 1-A Imagery. *Information*, 14(1), 10. <https://doi.org/10.3390/info14010010>
- Seminar, K. B., Imantho, H., Sudradjat, Yahya, S., Munir, S., Kaliana, I., Mei Haryadi, F., Noor Baroroh, A., Supriyanto, Handoyo, G. C., Kurnia Wijayanto, A., Ijang Wahyudin, C., Liyantono, Budiman, R., Bakir Pasaman, A., Rusiawan, D., & Sulastri. (2024). PreciPalm: An Intelligent System for Calculating Macronutrient Status and Fertilizer Recommendations for Oil Palm on Mineral Soils Based on a Precision Agriculture Approach. *Scientific World Journal*, 2024(1). <https://doi.org/10.1155/2024/1788726>